XML w bazach danych

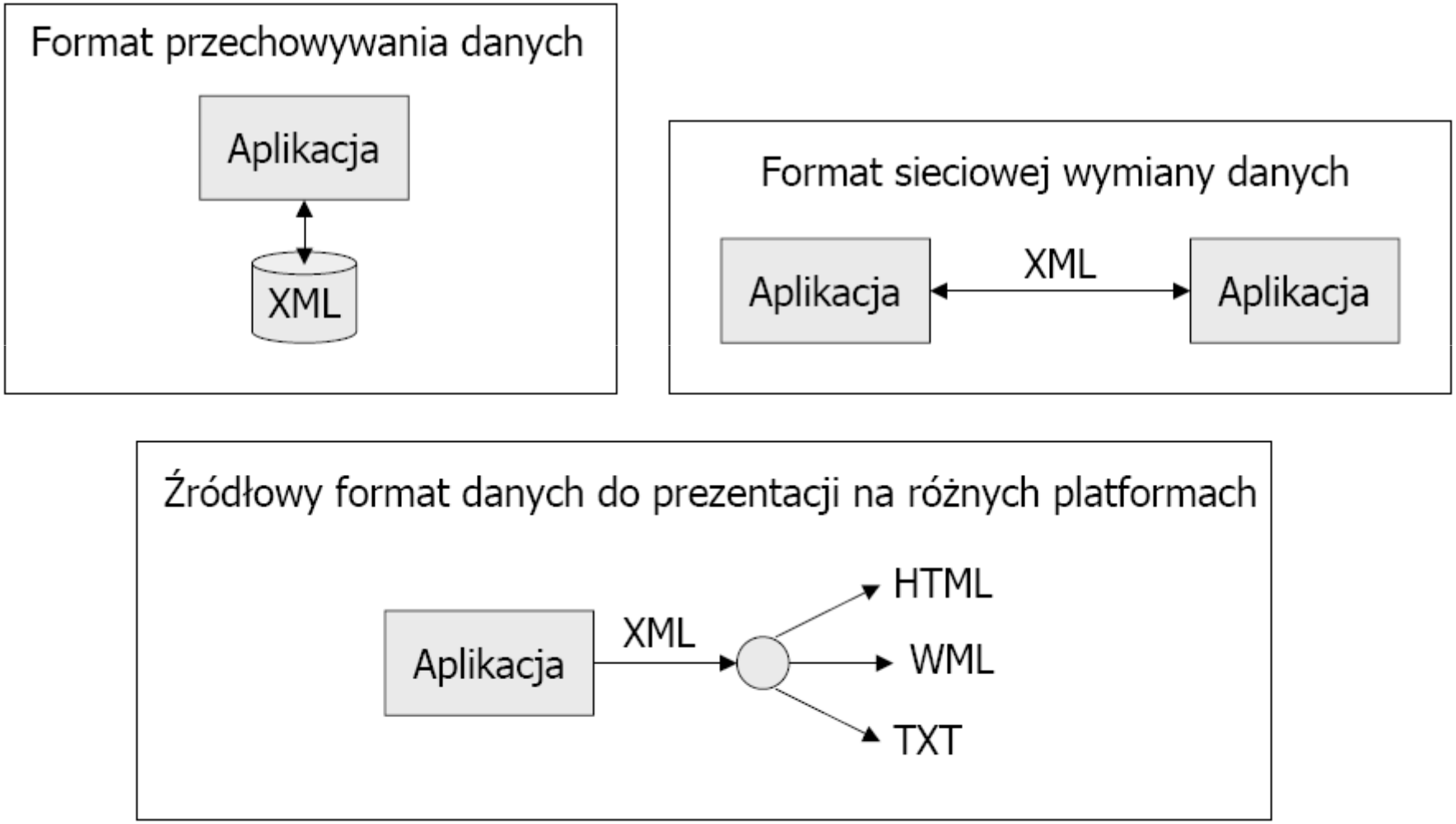
Plan

- Część 1
  - XML
  - Standard SQL-XML
  - Funkcje SQL-XML konstruujące obiekty XML (zadania)
- Część 2
  - XMLType (zadania)
  - XPath
  - Metody i funkcje SQL wykorzystujące XPath (zadania)
- Część 3
  - XQuery
  - Funkcje wykorzystujące XQuery (zadania)

Format XML

- Formalnie, **XML** stanowi podzbiór języka Standard Generalized Markup Language (**SGML**) (ISO 8879:1986)
- XML jest formatem służącym do opisywania strukturalnych danych, przeznaczonych do upowszechniania w sieci Internet
- Dokumenty XML posługują się znacznikami, zapisywanymi podobnie jak w HTML; jednak wszystkie znaczniki XML są definiowane przez programistę
- Dokumenty XML są łatwe w przetwarzaniu maszynowym; istnieje wiele bibliotek tzw. parserów XML, z których mogą korzystać programiści (Java, JavaScript, C++, itd.)

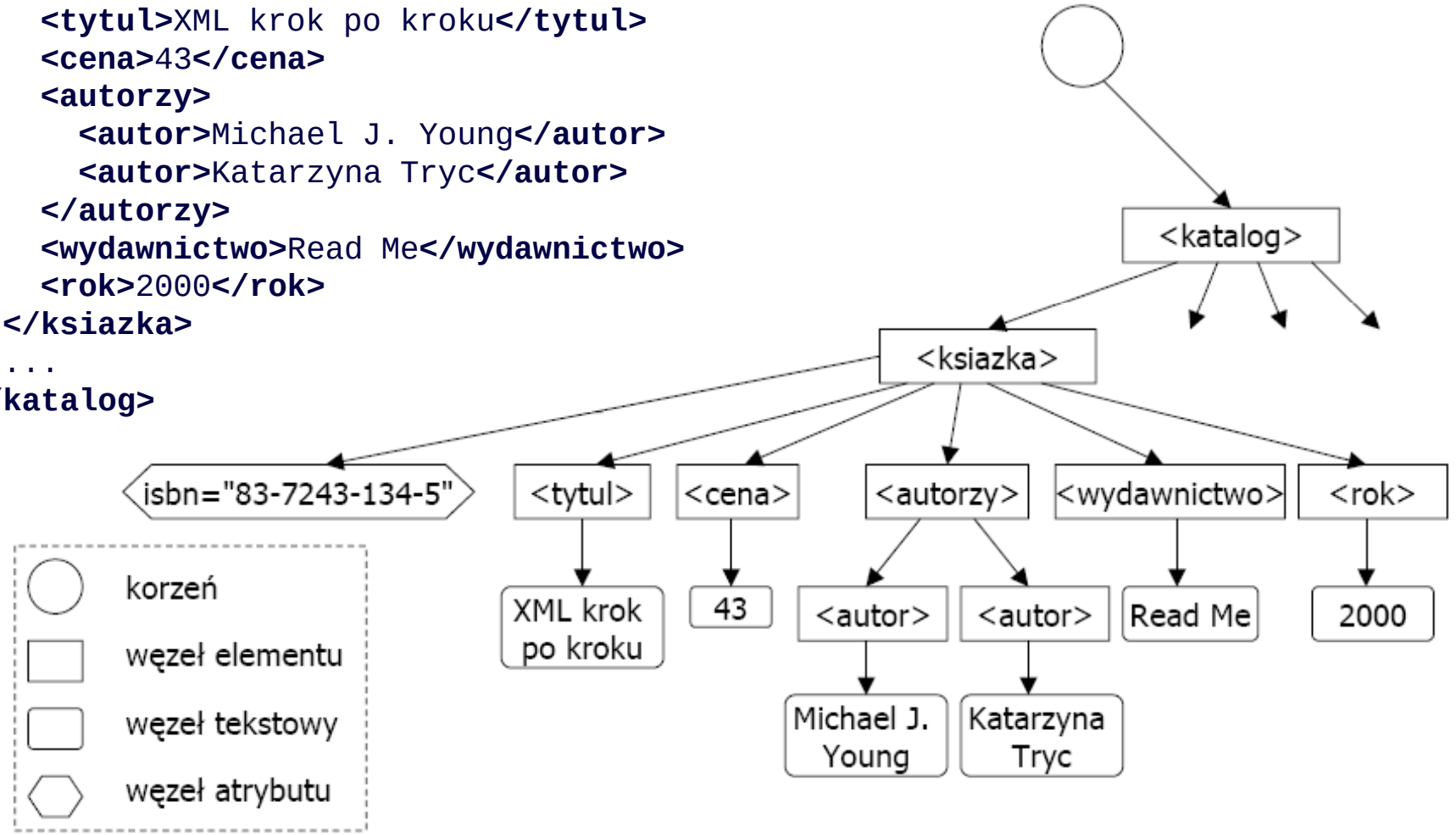
Wybrane zastosowania XML



XML

Przykładowy dokument, drzewo DOM

```
<katalog>
  <ksiazka isbn="83-7243-134-5">
    <tytul>XML krok po kroku</tytul>
    <cena>43</cena>
    <autorzy>
      <autor>Michael J. Young</autor>
      <autor>Katarzyna Tryc</autor>
    </autorzy>
    <wydawnictwo>Read Me</wydawnictwo>
    <rok>2000</rok>
  </ksiazka>
  ...
</katalog>
```



>> Standard SQL/XML <<

- Standard SQL/XML został zdefiniowany w postaci specyfikacji przygotowanej przez nieformalną grupę SQLX
- W pracach nad nim brali udział przedstawiciele takich firm jak: DataDirect Technologies, Hewlett-Packard, IBM, Microsoft, Oracle, Sybase i innych
- SQL/XML to ogólny standard dotyczący zastosowania SQL w ogólnie rozumianym przetwarzaniu dokumentów XML
- Jego pierwsze implementacje zostały stworzone przez Oracle, IBM i DataDirect Technologies
- Pod koniec roku 2003 SQL/XML stał się częścią standardu języka SQL. Został on przedstawiony w rozdziale 14. dokumentu *ISO/IEC 9075:2003 Information technology – Database languages – SQL* opisującego standard SQL:2003

Zakres standardu SQL/XML

- Mapowanie pomiędzy SQL (schemat i dane przechowywane w relacyjnej bazie danych) a XML. W szczególności dotyczy to mapowania pomiędzy:
  - zestawami znaków SQL a Unicode
  - tabelami, schematami i katalogami a dokumentami XML
  - identyfikatorami SQL a nazwami XML
  - typami danych SQL i XML Schema
  - wartościami SQL i XML
- Typ danych XML
- Funkcje

Funkcje SQL/XML

- Funkcje standardu SQL/XML to najbardziej popularny i najczęściej wykorzystywany jego zakres
- Można je podzielić na dwie podstawowe kategorie:
  - **funkcje tworzące obiekty XML** jako wynik polecenia SELECT, oraz
  - funkcje adresujące fragmenty dokumentów XML lub wykonujące zapytania na dokumentach XML i wykorzystywane w operacjach SQL, lub po prostu
    funkcje wykorzystujące zapytania XQuery do przetwarzania dokumentów XML

Funkcje tworzące obiekty XML (1/2)

- **XMLElement** – tworzy **element** XML
- **XMLAttributes** – dodaje do elementu XML utworzonego za pomocą funkcji XMLElement **atrybuty**
- **XMLForest** – tworzy **las** elementów XML
- **XMLConcat** – **łączy** wiele fragmentów XML w jeden
- **XMLAgg** – **funkcja grupowa** łącząca wiele elementów XML występujących na poziomie wierszy w jeden fragment na poziomie grupy

W Oracle Database 10g release 2 zostały dodane następujące funkcje:

- **XMLPI** – tworzy **instrukcję przetwarzania**
- **XMLComment** – tworzy **komentarz** XML
- **XMLRoot** – dodaje **nagłówek** dokumentu XML
- **XMLSerialize** – **konwertuje** dokument XML lub jego fragment na ciąg znaków lub obiekt CLOB
- **XMLParse** – służy do **parsowania** ciągu znaków traktowanego jako dokument XML lub jego fragment na obiekt typu XMLType

Funkcje tworzące obiekty XML (2/2)

- Oprócz wcześniej wymienionych funkcji wchodzących w skład standardu SQL/XML, Oracle proponuje również własne funkcje, do których należą przede wszystkim:
  - **XMLSequence** – tworzy **tablicę** obiektów XMLType dla przykładu na podstawie danych pobieranych przez kursor
  - **XMLColAttVal** – funkcja, która tworzy **las** elementów XML o nazwie **COLUMN**
  - **XMLCDATA** – tworzy obiekt **CDATA**, (10g release 2)
  - **SYS_XMLGen** – tworzy **dokument** XML dla każdego wiersza zapytania.
  - **SYS_XMLAgg** – **funkcja grupowa**, tworzy **dokument** XML oparty na zwartości wierszy w grupie.

Funkcja XMLElement

- Pobiera nazwę elementu oraz opcjonalną kolekcję atrybutów i obiektów tworzących zawartość elementu i tworzy z nich instancję typu XMLType
- Składnia:

```
XMLELEMENT ( [NAME] identifier
             [, XML_attributes_clause]
             [, value_expr [, value_expr]... ] )
```

```
XMLATTRIBUTES (value_expr [AS c_alias]
                [, value_expr [AS c_alias]...)
```

Funkcja XMLElement

przykłady (1/4)

```
SELECT id_prac, XMLELEMENT ( "PRACOWNIK", nazwisko ) AS "wynik"
FROM pracownicy
```

```
   ID_PRAC wynik
---------- ------------------------------------
       100 <PRACOWNIK>WEGLARZ</PRACOWNIK>
       110 <PRACOWNIK>BLAZEWICZ</PRACOWNIK>
       120 <PRACOWNIK>SLOWINSKI</PRACOWNIK>
. . .
```

```
SELECT XMLELEMENT ( "PRACOWNIK", XMLAttributes(id_prac), nazwisko ) AS "wynik"
FROM pracownicy
```

```
wynik
--------------------------------------------------
<PRACOWNIK ID_PRAC="100">WEGLARZ</PRACOWNIK>
<PRACOWNIK ID_PRAC="110">BLAZEWICZ</PRACOWNIK>
<PRACOWNIK ID_PRAC="120">SLOWINSKI</PRACOWNIK>
<PRACOWNIK ID_PRAC="130">BRZEZINSKI</PRACOWNIK>
. . .
```

Funkcja XMLElement

przykłady (2/4)

```
SELECT XMLElement ( "PRACOWNIK",
                    XMLAttributes(id_prac as "id"),
                    XMLElement ("NAZWISKO", nazwisko ),
                    ' pracuje jako '||etat||' i zarabia ',
                    XMLElement ("PLACA", placa_pod+nvl(placa_dod,0))
                  ) AS "wynik"
FROM pracownicy
```

aliasy

zagnieżdżenia

mieszana zawartość

```
<PRACOWNIK id="100">
  <NAZWISKO>WEGLARZ</NAZWISKO>
  pracuje jako DYREKTOR i zarabia   <PLACA>2150.5</PLACA>
</PRACOWNIK>

<PRACOWNIK id="110">
  <NAZWISKO>BLAZEWICZ</NAZWISKO>
  pracuje jako PROFESOR i zarabia   <PLACA>1560</PLACA>
</PRACOWNIK>

<PRACOWNIK id="120">
  <NAZWISKO>SLOWINSKI</NAZWISKO>
  pracuje jako PROFESOR i zarabia   <PLACA>1070</PLACA>
</PRACOWNIK>
. . .
```

Funkcja XMLElement

przykłady (3/4)

```
SELECT XMLELEMENT ( "PRACOWNIK",
                    XMLAttributes(id_prac, placa_pod, placa_dod),
                    nazwisko ) AS "wynik"
FROM pracownicy
```

Wartość NULL argumentu XMLAttributes skutkuje brakiem danego atrybutu w tworzonym elemencie

```
wynik
-----------------------------------------------------------------------------
<PRACOWNIK ID_PRAC="100" PLACA_POD="1730" PLACA_DOD="420.5">WEGLARZ</PRACOWNIK>
<PRACOWNIK ID_PRAC="110" PLACA_POD="1350" PLACA_DOD="210">BLAZEWICZ</PRACOWNIK>
<PRACOWNIK ID_PRAC="120" PLACA_POD="1070">SLOWINSKI</PRACOWNIK>
. . .
```

```
SELECT XMLElement (
         "PRACOWNIK",
         XMLAttributes(id_prac as "id"),
         XMLElement ("NAZWISKO", nazwisko ),
         XMLElement ("DODATEK", placa_dod)
         ) AS "wynik"
FROM pracownicy
```

Wartość NULL argumentu XMLElement powoduje utworzenie pustego elementu

```
wynik
--------------------------------
<PRACOWNIK id="100">
  <NAZWISKO>WEGLARZ</NAZWISKO>
  <DODATEK>420.5</DODATEK>
</PRACOWNIK>
. . .
<PRACOWNIK id="120">
  <NAZWISKO>SLOWINSKI</NAZWISKO>
  <DODATEK/>
</PRACOWNIK>
. . .
```

Funkcja XMLElement

przykłady – obiektowe (4/4)

```
CREATE TYPE szef_t AS
  OBJECT(
     nazwisko VARCHAR2(20)
  );
/
```

```
CREATE OR REPLACE TYPE prac_t AS
  OBJECT(
    "@prac_id" NUMBER,
     nazwisko VARCHAR2(20),
     szef     szef_t
  );
/
```

```
CREATE TYPE praclist_t AS
       TABLE OF prac_t;
/

CREATE TYPE zesp_t AS OBJECT(
    "@zesp_id" NUMBER,
     nazwa VARCHAR2(20),
     pracownicy praclist_t
  );
/
```

```
SELECT XMLElement("ZESPOL",
       XMLATTRIBUTES (
         'http://www.w3.org/2001/XMLSchema' AS "xmlns:xsi",
         'zespol.xsd' AS "xsi:nonamespaceSchemaLocation" ),
       zesp_t(id_zesp, nazwa,
       CAST(MULTISET
            (SELECT p.id_prac, p.nazwisko,
             SZEF_T(s.nazwisko) szef
             FROM   pracownicy p, pracownicy s
             WHERE  p.id_zesp = z.id_zesp
             AND    p.id_szefa = s.id_prac(+))
                     AS   praclist_t))) zespol
from zespoly z
```

```
<ZESPOL xmlns:xsi="http://www.w3.org/2001/XMLSchema"
        xsi:nonamespaceSchemaLocation="zespol.xsd">
  <ZESP_T zesp_id="10">
    <NAZWA>ADMINISTRACJA</NAZWA>
    <PRACOWNICY>
      . . .
      <PRAC_T prac_id="180">
        <NAZWISKO>MAREK</NAZWISKO>
        <SZEF>
          <NAZWISKO>WEGLARZ</NAZWISKO>
        </SZEF>
      </PRAC_T>
    </PRACOWNICY>
  </ZESP_T>
</ZESPOL> . . .
```

Funkcja XMLForest

- Tworzy las elementów z podanej listy argumentów. Argumenty mogą być wyrażeniami z opcjonalnymi aliasami
- Składnia:

```
XMLFOREST (value_expr [AS c_alias] [, value_expr [AS c_alias]... ])
```

XMLForest

przykłady

```
SELECT XMLElement (
        "PRACOWNIK",
        XMLAttributes(id_prac as "id"),
        XMLForest ( nazwisko as "NAZWISKO",
                    placa_dod as "DODATEK")
        ) AS "wynik"
FROM pracownicy;
```

```
wynik
-----------------------------------
<PRACOWNIK id="100">
  <NAZWISKO>WEGLARZ</NAZWISKO>
  <DODATEK>420.5</DODATEK>
</PRACOWNIK>

<PRACOWNIK id="110">
  <NAZWISKO>BLAZEWICZ</NAZWISKO>
  <DODATEK>210</DODATEK>
</PRACOWNIK>

<PRACOWNIK id="120">
  <NAZWISKO>SLOWINSKI</NAZWISKO>
</PRACOWNIK>
. . .
```

Wartość NULL powoduje pominięcie danego elementu

```
SELECT XMLElement (
        "PRACOWNIK",
        XMLAttributes(id_prac as "id"),
        XMLForest ( nazwisko as "NAZWISKO",
          XMLForest( placa_pod as "PODSTAWA",
                     placa_dod as "DODATEK")
            as "ZAROBKI"
        )) AS "wynik"
FROM pracownicy
```

```
wynik
---------------------------------
<PRACOWNIK id="100">
  <NAZWISKO>WEGLARZ</NAZWISKO>
  <ZAROBKI>
    <PODSTAWA>1730</PODSTAWA>
    <DODATEK>420.5</DODATEK>
  </ZAROBKI>
</PRACOWNIK>
. . .
<PRACOWNIK id="120">
  <NAZWISKO>SLOWINSKI</NAZWISKO>
  <ZAROBKI>
    <PODSTAWA>1070</PODSTAWA>
  </ZAROBKI>
</PRACOWNIK>
. . .
```

XMLConcat

- Konkatenuje argumenty tworząc jeden fragment XML
- XMLConcat posiada dwie postacie
  - Jedna jako parametr pobiera obiekt typu XMLSequenceType i zwraca pojedynczą instancję typu XMLType będącą konkatenacją wszystkich elementów w XMLSequenceType
  - Druga forma konkatenuje dowolną liczbę atrybutów typu XMLType w jedną instancję XMLType
- Składnia:

```
XMLCONCAT ( XMLType_instance [, XMLType_instance]... )
```

XMLConcat
przykłady

```
SELECT XMLConcat (
         XMLElement("SZEF", S.NAZWISKO ),
         XMLElement("PODWLADNY", P.NAZWISKO)
) AS "wynik"
FROM  pracownicy p, pracownicy s
WHERE p.id_szefa = s.id_prac;
```

```
wynik
------------------------------------
<SZEF>WEGLARZ</SZEF>
<PODWLADNY>BLAZEWICZ</PODWLADNY>

<SZEF>WEGLARZ</SZEF>
<PODWLADNY>SLOWINSKI</PODWLADNY>

<SZEF>WEGLARZ</SZEF>
<PODWLADNY>BRZEZINSKI</PODWLADNY>
. . .
```

```
select XMLConcat(XMLSequence(
 Cursor(SELECT NAZWISKO,
               PLACA_POD,
               PLACA_DOD
        FROM pracownicy))) Wynik
from dual;
```

```
WYNIK
--------------------------------
<ROW>
  <NAZWISKO>WEGLARZ</NAZWISKO>
  <PLACA_POD>1730</PLACA_POD>
  <PLACA_DOD>420.5</PLACA_DOD>
</ROW>
<ROW>
  <NAZWISKO>BLAZEWICZ</NAZWISKO>
  <PLACA_POD>1350</PLACA_POD>
  <PLACA_DOD>210</PLACA_DOD>
</ROW>
. . .
<ROW>
  <NAZWISKO>HAPKE</NAZWISKO>
  <PLACA_POD>480</PLACA_POD>
  <PLACA_DOD>90</PLACA_DOD>
</ROW>

1 row selected
```

XMLAgg

- Jest funkcją grupującą.
- Tworzy las z kolekcji elementów XML – jeden dla każdej grupy rekordów
- Przed utworzeniem lasu elementów dane źródłowe można posortować
- Składnia:

```
XMLAGG (XMLType_instance [ ORDER BY sort_list ] )
```

```
ORDER BY [list of: expr [ASC|DESC] [NULLS {FIRST|LAST} ] ]
```

XMLAgg
przykłady (1/2)

```
SELECT XMLElement ( "ZESPOL",
         XMLAttributes( z.id_zesp AS "id_zesp" ),
         XMLElement( "NAZWA", z.nazwa),
         XMLElement( "PRACOWNICY",
           XMLAgg( XMLElement ( "PRACOWNIK", p.nazwisko ))
                 ) ) AS "wynik"
FROM pracownicy p, zespoly z
WHERE p.id_zesp = z.id_zesp
GROUP BY z.id_zesp, z.nazwa;
```

```
wynik
----------------------------------------
<ZESPOL id_zesp="10">
  <NAZWA>ADMINISTRACJA</NAZWA>
  <PRACOWNICY>
    <PRACOWNIK>WEGLARZ</PRACOWNIK>
    <PRACOWNIK>MAREK</PRACOWNIK>
  </PRACOWNICY>
</ZESPOL>
. . .
<ZESPOL id_zesp="40">
  <NAZWA>ALGORYTMY</NAZWA>
  <PRACOWNICY>
    <PRACOWNIK>BLAZEWICZ</PRACOWNIK>
  </PRACOWNICY>
</ZESPOL>
```

XMLAgg
przykłady (2/2)

```
SELECT XMLElement ( "ZESPOL",
         XMLAttributes( z.id_zesp AS "id_zesp" ),
         XMLElement( "NAZWA", z.nazwa),
         XMLElement( "PRACOWNICY",
           XMLAgg(
             XMLElement ( "PRACOWNIK",
               XMLElement ( "NAZWISKO", p.nazwisko ),
               XMLElement ( "PODWLADNI",
                 ( SELECT XMLAgg(
                             XMLElement( "PODWLADNY",
                               XMLAttributes(d.NAZWISKO AS "NAZWISKO")))
                    FROM  pracownicy d
                    WHERE d.id_szefa = p.id_prac ) )
                ) ) ) ) AS "wynik"
FROM pracownicy p, zespoly z
WHERE p.id_zesp = z.id_zesp
GROUP BY z.id_zesp, z.nazwa;
```

```
wynik
----------------------------------------
<ZESPOL id_zesp="10">
  <NAZWA>ADMINISTRACJA</NAZWA>
  <PRACOWNICY>
    <PRACOWNIK>
      <NAZWISKO>WEGLARZ</NAZWISKO>
      <PODWLADNI>
        <PODWLADNY NAZWISKO="BLAZEWICZ"/>
        <PODWLADNY NAZWISKO="SLOWINSKI"/>
        <PODWLADNY NAZWISKO="BRZEZINSKI"/>
        <PODWLADNY NAZWISKO="MAREK"/>
      </PODWLADNI>
    </PRACOWNIK>
    <PRACOWNIK>
      <NAZWISKO>MAREK</NAZWISKO>
      <PODWLADNI/>
    </PRACOWNIK>
  </PRACOWNICY>
</ZESPOL>
. . .
```

XMLColAttVal

- Działa podobnie jak XMLForest.
  Różnica polega na tym że:
  - Wynikiem jest las elementów <column>.
  - Elementy <column> posiadają atrybut name którego wartość pochodzi z nazwy (aliasu) wyrażenia.
  - Zawartość elementu <column> jest wyznaczana na podstawie wartości wyrażenia.
- Cel: uniknięcie konwersji nazw kolumn w SQL do nazw elementów XML
- Składnia:

```
XMLCOLATTVAL (value_expr [AS c_alias]
                [, value_expr [AS c_alias]...)
```

XMLColAttVal

przykłady

```
SELECT XMLElement (
         "PRACOWNIK",
         XMLAttributes(id_prac as "id"),
         XMLColAttVal ( nazwisko as "NAZWISKO",
                        placa_dod as "DODATEK")
         ) AS "wynik"
FROM pracownicy;
```

```
wynik
-----------------------------------------
<PRACOWNIK id="100">
  <column name="NAZWISKO">WEGLARZ</column>
  <column name="DODATEK">420.5</column>
</PRACOWNIK>

<PRACOWNIK id="110">
  <column name="NAZWISKO">BLAZEWICZ</column>
  <column name="DODATEK">210</column>
</PRACOWNIK>

<PRACOWNIK id="120">
  <column name="NAZWISKO">SLOWINSKI</column>
  <column name="DODATEK"/>
</PRACOWNIK>
. . .
```

Wartość NULL powoduje utworzenie pustego elementu

```
SELECT XMLElement (
      "PRACOWNIK",
      XMLAttributes(id_prac as "id"),
      XMLColAttVal ( nazwisko as "NAZWISKO",
        XMLColAttVal( placa_pod as "PODSTAWA",
                      placa_dod as "DODATEK")
          as "ZAROBKI"
      )) AS "wynik"
 FROM pracownicy
```

```
wynik
-----------------------------------------
<PRACOWNIK id="100">
  <column name="NAZWISKO">WEGLARZ</column>
  <column name="ZAROBKI">
    <column name="PODSTAWA">1730</column>
    <column name="DODATEK">420.5</column>
  </column>
</PRACOWNIK>

<PRACOWNIK id="110">
  <column name="NAZWISKO">BLAZEWICZ</column>
  <column name="ZAROBKI">
    <column name="PODSTAWA">1350</column>
    <column name="DODATEK">210</column>
  </column>
</PRACOWNIK>
. . .
```

Przykłady funkcji tworzących obiekty XML (1/2)


```
SQL> select XMLElement("PRAC",
  2                    XMLAttributes(ZATR),
  3                    XMLForest(NAZWA, PLACA))
  4  from PR;

XMLELEMENT("PRAC",XMLATTRIBUTES(ZATR),XMLFOREST(NAZWA,PLACA))
--------------------------------------------------------------------------------
<PRAC ZATR="2006-02-20">
  <NAZWA>KRZYSZTOF NOWAK</NAZWA>
  <PLACA>1000</PLACA>
</PRAC>
<PRAC ZATR="2006-02-10">
  <NAZWA>TOMASZ KOWALSKI</NAZWA>
  <PLACA>2000</PLACA>
</PRAC>
```

```
SQL> select XMLElement("PRACOWNICY",
  2           XMLAgg(
  3             XMLConcat(
  4               XMLForest(NAZWA,PLACA),
  5               XMLElement("Data_Zatr",ZATR)
  6             ) ORDER BY NAZWA DESC ),
  7           XMLPI(NAME "xmldb", 'end_of_document') wynik
  8         ) wynik
  9  from PR;

WYNIK
--------------------------------------------------------------------------------
<PRACOWNICY>
          <NAZWA>TOMASZ KOWALSKI</NAZWA>
          <PLACA>2000</PLACA>
          <Data_Zatr>2006-02-10</Data_Zatr>
          <NAZWA>KRZYSZTOF NOWAK</NAZWA>
          <PLACA>1000</PLACA>
          <Data_Zatr>2006-02-20</Data_Zatr>
          <?xmldb end_of_document?>
</PRACOWNICY>
```

Przykłady funkcji tworzących obiekty XML (2/2)


```
SQL> select XMLRoot(
  2           XMLElement("DOKUMENT",
  3             sys_xmlgen('TRESC',XMLFormat('MALY')),
  4             XMLComment('mały dokument')),
  5           VERSION '1.1' ) wynik
  6  from dual;

WYNIK
--------------------------------------------------------
<?xml version="1.1"?>
<DOKUMENT>
  <MALY>TRESC</MALY>
  <!--mały dokument-->
</DOKUMENT>
```

```
select XMLParse(CONTENT
          'Tu byłem <OSOBA>Mały</OSOBA>' WELLFORMED).isFragment()
from dual;
```

```
select LENGTH(
         XMLSerialize(DOCUMENT XMLType('<MALY>TRESC</MALY>') AS VARCHAR(20)))
from dual;
```

Koniec części 1.

>> XMLType <<

- Typ danych oferujący wewnętrzne mechanizmy przetwarzania danych XML w bazie danych
- Wystąpienia typu XMLType reprezentują dokumenty XML w SQL
- XMLType może być również używany w PL/SQL
- Interfejs typu XMLType zawiera metody do manipulacji zawartością XML
- Fukcjonalność typu XMLType jest dostępna z poziomu języków PL/SQL i Java
- XMLType może być składowany na dwa sposoby:
  - Jako duży obiekt: CLOB (domyślnie)
  - W postaci strukturalnej: zdekomponowany do tabel

Kiedy używać XMLType?

- Gdy istnieje konieczność kierowania zapytań do dokumentów XML lub ich części
- W celu zagwarantowania, że przetwarzane dane tekstowe rzeczywiście maja postać XML (kontrola typu)
- Dla zwiększenia wydajności ewaluacji wyrażeń XPath
- Aby umożliwić zakładanie indeksów na wyrażeniach XPath
- Aby odseparować aplikacje od szczegółów dotyczących składowania dokumentów XML w bazie danych

Tabele i atrybuty typu XMLType

- Dokumenty XML mogą być składowane w postaci wystąpień typu XMLType:
  - W kolumnach typu XMLType
  - W tabelach typu XMLType

```
CREATE TABLE produkty_xml (
kod NUMBER(11),
opis XMLType
);
```

```
CREATE TABLE ksiazki_xml
OF XMLType;
```

XMLType w poleceniach SQL (1/2)

- Obiekt typu XMLType można utworzyć z danych typu VARCHAR lub CLOB konstruktorem XMLType()

```
INSERT INTO produkty_xml
VALUES (67653829370, XMLType(
        '<produkt>
                <nazwa>Antena dachowa</nazwa>
                <symbol>1709765</symbol>
                <cena>85</cena>
        </produkt>'));
```

```
insert into ksiazki_xml
values (XMLType('
        <ksiazka>
                <tytul>
                ABC XML
                </tytul>
        </ksiazka>'));
```

```
SQL> select opis from produkty_xml;
OPIS
------------------------------------
<produkt>
        <nazwa>Antena dachowa</nazwa>
        <symbol>1709765</symbol>
        <cena>85</cena>
</produkt>
```

Odczyt wartości XMLType z tabeli typu XMLType

```
select value(p)
from ksiazki_xml p;
```

XMLType w poleceniach SQL (2/2)

- Operacje na całych obiektach typu XMLType realizowane są tradycyjnymi poleceniami SQL

```
UPDATE produkty_xml
SET opis = XMLType(
        '<produkt>
                <nazwa>Antena dachowa</nazwa>
                <symbol>1709765</symbol>
                <cena>100</cena>
        </produkt>')
WHERE kod = 67653829370;
```

```
DELETE FROM produkty_xml
WHERE kod = 67653829370;
```

Tworzenie wartość XMLType

- Tworzenie wartości XMLType może odbywać się na kilka sposobów
  - Wykorzystując konstruktor XMLType. Z wartości
    - VARCHAR
    - CLOB
  - Wykorzystując funkcje SQL/XML
  - Ładując zawartość pliku XML

Tworzenie wartości XMLType funkcjami SQL/XML

- Dokumenty XML, które maja być umieszczone w kolumnach/tabelach typu XMLType można tworzyć funkcjami SQL/XML
- Wartością XMLType, składowanej w bazie danych, może stać się element, a nie las elementów (!)

```
INSERT INTO produkty_xml
SELECT kod, XMLElement(NAME "produkt",
               XMLForest(     nazwa "nazwa",
                              symbol "symbol",
                              cena "cena")
        )
FROM produkty_sql;
```

Ładowanie pliku XML do kolumny XMLType

```
CREATE TABLE myxml (doc_id NUMBER(6), doc XMLType);

CREATE DIRECTORY xmlsource AS '/oracle';

DECLARE
  xmldoc CLOB;
  file BFILE;
BEGIN
  dbms_lob.createTemporary(xmldoc,true);
  file := BFileName('XMLSOURCE','merlin.xml');
  dbms_lob.fileOpen(file);
  dbms_lob.loadFromFile(xmlDoc, file, dbms_lob.getLength(file));
  INSERT INTO myxml VALUES (1, XMLType.createxml(xmldoc));
  dbms_lob.fileClose(file);
END;
```

>> Język XPath <<

- XPath to specyfikacja języka służącego do adresowania, odczytywania i przeszukiwania drzew DOM dokumentów XML
- XPath odgrywa podobną rolę w stosunku do drzew DOM, jak język SQL w stosunku do relacyjnych baz danych
- XPath stosuje notację przypominającą ścieżki dostępu w systemach plików
- Wynikiem ewaluacji wyrażenia XPath jest zbiór węzłów spełniających warunki selekcji
- XPath pozwala stosować dwa rodzaje zapisu wyrażeń: skrócony i pełny; rodzaje te mogą być mieszane

Wyrażenia XPath

- Wyrażenie ścieżkowe XPath składa się z tzw. kroków rozdzielonych ukośnikami
- W pełnym zapisie, każdy z kroków może składać się z:
  - **specyfikatora współrzędnych** (ang. *axis*), służącego do określenia miejsca w drzewie, począwszy od którego wyszukiwane będą węzły
  - **testu węzła** (ang. *node test*), służącego do określenia, które węzły są wyszukiwane w obszarze drzewa określonym przez specyfikator współrzędnych
  - **predykatów**, dodatkowo zawężających test węzła, powodujących wybór tylko tych węzłów, które spełniają podany warunek
- Każdy krok pełnego wyrażenia XPath zapisywany jest przy użyciu następującej notacji:

spec_współrzędnych::test_węzła[predykaty]

```
/descendant-or-self::katalog/child::ksiazka[attribute::isbn= "83-7197-786-7"]
```

Specyfikatory współrzędnych

specyfikator	skrót	znaczenie
self	.	obejmuje bieżący węzeł
preceding-sibling		obejmuje wszystkie węzły sąsiednie, które w dokumencie następują przed węzłem bieżącym
preceding		obejmuje wszystkie węzły, które w dokumencie następują przed węzłem bieżącym
parent	..	obejmuje węzeł bezpośrednio nadrzędny bieżącego węzła (ojciec)
following-sibling		obejmuje wszystkie węzły sąsiednie, które w dokumencie następują za węzłem bieżącym
following		obejmuje wszystkie węzły, które w dokumencie następują za węzłem bieżącym
descendant-or-self	//	obejmuje bieżący węzeł plus wszystkie węzły podrzędne
descendant		obejmuje wszystkie węzły podrzędne (syn, wnuk, itd.) bieżącego węzła
child	brak	obejmuje wszystkie węzły bezpośrednio podrzędne bieżącego węzła (dzieci)
attribute	@	obejmuje wszystkie atrybuty bieżącego węzła
ancestor-or-self		obejmuje bieżący węzeł plus wszystkie węzły nadrzędne
ancestor		obejmuje wszystkie węzły nadrzędne (ojciec, dziadek, itd.) bieżącego węzła

```
//katalog/ksiazka[@isbn= "83-7197-786-7"]
```

Testy węzłów, operatory i funkcje

nazwa	węzeł o podanej nazwie
@*	dowolny atrybut
*	dowolny element
text()	węzeł tekstowy
node()	dowolny węzeł

false()	zwraca fałsz logiczny
true()	zwraca prawdę logiczną
not()	negacja logiczna
position()	liczba porządkowa węzła wybieranego przez wyrażenie
name()	nazwa węzła wybieranego przez wyrażenie
last()	liczba porządkowa ostatniego węzła wybieranego przez wyrażenie
count()	liczba węzłów wybieranych przez wyrażenie
+, -, *, div, mod	operatory arytmetyczne
or, and	operatory logiczne
=, !=, <, >, <=, >=	porównania

Operatory i funkcje

Funkcje znakowe

translate()	dokonuje zamiany wszystkich wystąpień podanego podciągu
substring-before()	zwraca ciąg znaków znajdujący się przed wycinanym podciągiem
substring-after()	zwraca ciąg znaków znajdujący się za wycinanym podciągiem
substring()	ekstrakcja podciągu znaków
string-length()	długość tekstu
string()	konwersja do tekstu
starts-with()	sprawdzenie, czy tekst rozpoczyna się od podanego ciągu
contains()	test zawierania tekstów
concat()	konkatenacja tekstów

Funkcje liczbowe

sum()	suma zbioru wartości liczbowych
round()	zaokrąglenie
number()	konwersja do liczby
floor()	dolne domknięcie całkowite
ceiling()	górne domknięcie całkowite

Przykłady wyrażeń XPath (1/2)

- Wybierz książki, których cena nie przekracza 20 zł
  //ksiazka[cena<=20]
- Wybierz tytuły książek o cenach w przedziale 30-40 zł
  //ksiazka/tytul[../cena>30 and ../cena<40]
- Wybierz książki napisane przez więcej niż dwóch autorów
  //ksiazka[count(autorzy/autor)>2]
- Wybierz co drugą książkę
  //ksiazka[position() mod 2 = 1]
- Wybierz książki zawierające w tytule słowo XML
  //ksiazka[contains(tytul,"XML")]
- Wybierz autorów o imieniu "Stephen"
  //autor[starts-with(.,"Stephen")]
- Wybierz tytuły złożone z ponad 20 znaków
  //tytul[string-length(.)>20]

Przykłady wyrażeń XPath (2/2)

- Wybierz książki, których numer ISBN spełnia wzorzec ***7197******
  //ksiazka[substring(@isbn,4,4) = "7197"]
- Wybierz wszystkie węzły autorzy oraz wszystkie ich węzły potomne
  //autorzy/descendant-or-self::*
- Wybierz wszystkie książki, które w dokumencie znajdują się za książką o numerze ISBN "83-7197-786-7"
  //ksiazka[@isbn="83-7197-786-7"]/following::ksiazka
- Wybierz wszystkie atrybuty pierwszej książki
  //ksiazka[1]/attribute::*
- Wybierz cenę książki pt. "XML dla każdego"
  //ksiazka[tytul="XML dla każdego"]/child::cena

Funkcje SQL operujące na danych XMLType (1/2)

- **value()** – wykorzystywany do uzyskania wartości XMLType z tabel typu XMLType
- **getStringVal()** – przekształca dokument XMLType w VARCHAR
- **getClobVal()** – przekształca dokument XMLType w CLOB
- **existsNode()** – sprawdza istnienie określonego węzła wskazanego za pomocą **XPath**
- **extract()** – zwraca fragment wskazany wyrażeniem **XPath**
- **extractValue()** – zwraca wartość skalarną odpowiadającą wynikowi wartościowania wyrażenia **XPath**
- **XMLTransform()** - Dokonuje transformacji dokumentu XML w oparciu o podany arkusz stylów XSL

Większość z funkcji operujących na XMLType jest dostępnych jako funkcje SQL i jako metody typu XMLType

Funkcje SQL operujące na danych XMLType (2/2)

- **updateXML()** – modyfikuje wybrane węzły w obiekcie XMLType
- **insertChildXML()** – dodaje węzeł podrzędny w stosunku do węzła kontekstu
- **insertXMLbefore()** – dodaje węzeł przed danym węzłem
- **appendChildXML()** – dodaje węzeł podrzędny – na samym końcu
- **deleteXML()** – usuwa węzeł

getStringVal() i getClobVal()

```
select DBMS_LOB.SUBSTR(p.opis.getClobVal(),7,2)
FROM produkty_xml p;

DBMS_LOB.SUBSTR(P.OPIS.GETCLOBVAL(),7,2)
----------------------------------------
produkt
```

```
select length(p.opis.getStringVal()|| ' - i co teraz?')
FROM produkty_xml p;

LENGTH(P.OPIS.GETSTRINGVAL()||'-ICOTERAZ?')
-------------------------------------------
                                        112
```

Funkcja existsNode()

- Sprawdza czy wartościowanie danego wyrażenia XPath względem wartości XMLType zwraca choć jeden element lub węzeł tekstowy
- Zwraca wartość liczbową 1 lub 0

```
SELECT p.kod
FROM produkty_xml p
WHERE p.opis.existsNode('/produkt[cena=85]') = 1;
```

```
SELECT kod
FROM produkty_xml
WHERE existsNode(opis, '/produkt[cena=85]') = 1;
```

```
DELETE FROM produkty_xml p
WHERE p.opis.existsNode('/produkt[cena=85]') = 1;
```

Funkcja extract()

- Zwraca fragment wskazany wyrażeniem XPath jako:
  - Pojedynczy węzeł
  - Zbiór węzłów
  - Wartość tekstową
- Gdy XPath zwraca zbiór pusty, wynikiem **extract()** jest NULL

```
SQL> SELECT p.extract('//autor')
     FROM ksiazki_xml p;

P.EXTRACT('//AUTOR')
--------------------------------
<autor>Stephen Forte</autor>
<autor>Thomas Howe</autor>
<autor>Kurt Wall</autor>

<autor>Paul Cassel</autor>
<autor>Craig Eddy</autor>
<autor>Jon Price</autor>
...
```

Autorzy pierwszej książki

Autorzy drugiej książki

```
select p.opis.extract('//symbol/text()')
from  produkty_xml p;
```

```
P.OPIS.EXTRACT('//SYMBOL/TEXT()')
---------------------------------
1709765

170976511-22
```

Funkcja extractValue()

- Zwraca wartość skalarną odpowiadającą wynikowi wartościowania wyrażenia XPath
- Może zastąpić konstrukcję **extract()** z ścieżką zakończoną na poziomie węzła text()
- Zwraca wartość tylko jednego węzła – w przypadku dopasowania przez wyrażenie XPath wielu węzłów zgłaszany jest błąd

```
select extractValue(p.opis,'//cena')
from  produkty_xml p;

EXTRACTVALUE(P.OPIS,'//CENA')
------------------------------
100
85
```

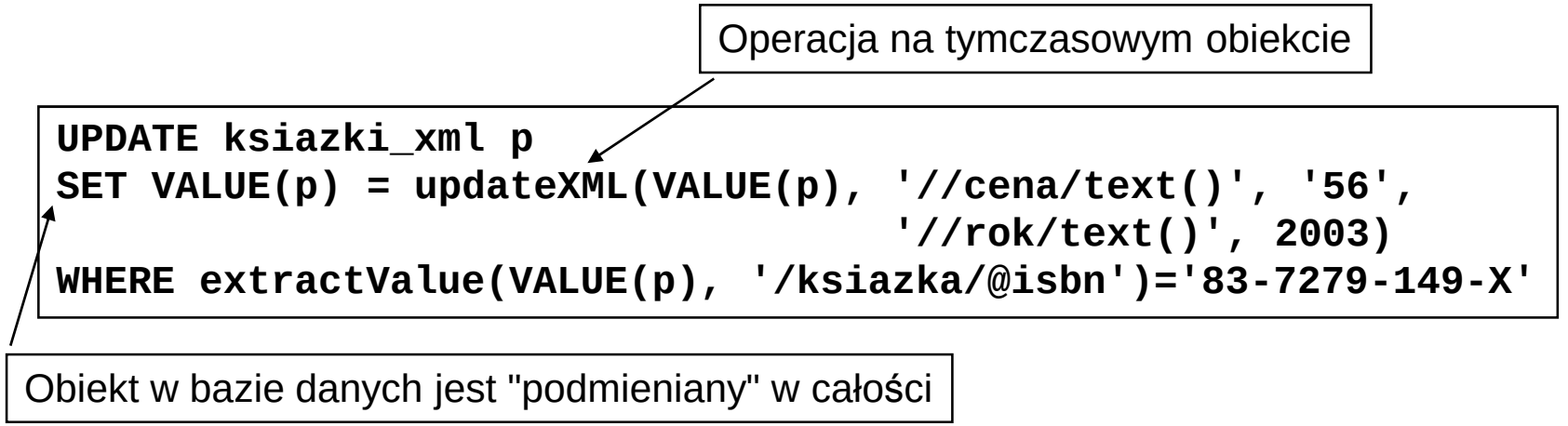
```
SELECT extractValue(value(p),
         '//cena/text()')
FROM ksiazki_xml p;

EXTRACTVALUE(VALUE(P),'//CENA/TEXT()')
-----------------------------------
79
65
59
```

```
select extractvalue(p.opis,'//symbol')
from  produkty_xml p;
ERROR:
ORA-19025: EXTRACTVALUE zwraca wartość
tylko jednego węzła
```

Funkcja updateXML()

- Modyfikuje wybrane węzły w obiekcie XMLType
- Działa na instancjach XMLType w pamięci
  - Tymczasowych obiektach tworzonych w pamięci w ramach przetwarzania zapytań
- Modyfikacje w bazie danych dokonywane poleceniem UPDATE
  - Podając wartości w klauzuli SET można wykorzystać funkcję **updateXML()**



updateXML() - Wielokrotna modyfikacja

- Jako argumenty **updateXML()** można podać kilka par: wyrażenie XPath, wartość
- Umożliwia to modyfikację kilku węzłów w jednej operacji
- Dopuszczalne jest również zmodyfikowanie tego samego węzła kilkukrotnie:
  - Wyrażenia **XPath** wartościowane od lewej do prawej
  - Każde kolejne wyrażenie **XPath** działa na wynikach poprzedniego

```
UPDATE ksiazki_xml p
SET VALUE(p) = updateXML(VALUE(p), '//cena/text()', '56',
                                   '//ksiazka/rok/text()', 2004)
WHERE extractValue(VALUE(p), '/ksiazka/@isbn')='83-7279-149-X';
```

```
UPDATE ksiazki_xml p
SET VALUE(p) = updateXML(VALUE(p), '//cena/text()', '56',
                                   '//ksiazka/rok',
                                   XMLType.createxml('<rok>2054</rok>'))
WHERE extractValue(VALUE(p), '/ksiazka/@isbn')='83-7279-149-X'
```

updateXML w zapytaniach i perspektywach

```
SQL> SELECT updateXML(VALUE(p), '//cena/text()', 0)
     FROM ksiazki_xml p
     WHERE extractValue(VALUE(p), '/ksiazka/@isbn')='83-7279-149-X';
UPDATEXML(VALUE(P),'//CENA/TEXT()',0)
----------------------------------------
<ksiazka isbn="83-7279-149-X">
         <tytul>Dane w sieci WWW</tytul>
         <cena>0</cena>
         . . .
</ksiazka>
```

```
create view produkty_bez_ceny as
select updatexml(p.opis,'//cena',NULL) produkt
from  produkty_xml p;
```

```
SQL> select * from produkty_bez_ceny;

PRODUKT
------------------------------------
<produkt>
  <nazwa>Antena dachowa</nazwa>
  <symbol>1709765</symbol>
  <cena/>
</produkt>
. . .
```

Funkcja insertChildXML

- Parametry funkcji insertChildXML to:
  - target-data (XMLType) – Dokument XML zawierający węzeł nadrzędny.
  - parent-xpath (VARCHAR2) – Wyrażenie XPath 1.0, które wskazuje elementy nadrzędne wewnątrz których mają zostać dodane węzły podrzędne.
  - child-name (VARCHAR2) – nazwa dodawanego węzła (atrybutu lub elementu). Nazwa atrybutu poprzedzona jest znakiem @
  - child-data (XMLType lub VARCHAR2) – dane XML które mają zostać umieszczone wewnątrz węzłów wskazywanych przez parent-xpath
    - jeśli wstawiany jest element wówczas typem parametru powinien być XMLType. Można wstawić wiele węzłów na raz, ważne aby wszystkie elementy na poziomie korzenia posiadały tę samą nazwę co w parametrze child-name.
    - jeśli wstawiany element jest atrybutem wówczas wartością parametru powinien być ciąg znaków – zawartość wstawianego atrybutu.
  - namespace (VARCHAR2, optional) – przestrzeń nazw dla parametrów parent-xpath i child-data

```
UPDATE ksiazki_or_m_xsd
  SET pozycja =
        insertChildXML(pozycja,
           '/ksiazka/autorzy',
           'autor',
           XMLType('<autor>Krzysztof J.</autor>'));
```

Funkcja insertXMLbefore

- Parametry funkcji insertXMLbefore to:
  - target-data (XMLType) – Dokument XML zawierający dokument, do którego zostaną dodane nowe węzły.
  - successor-xpath (VARCHAR2) – Wyrażenie XPath 1.0, które wskazuje węzły (oprócz atrybutów), przed którymi zostaną dodane nowe węzły.
  - XML-data (XMLType) – Dane XMLType, które zostaną wstawione jako węzły poprzedzające węzły wskazywane przez successor-xpath.
  - namespace (optional, VARCHAR2) – przestrzeń nazw dla parametru successor-xpath.

```
UPDATE ksiazki_or_m_xsd
  SET pozycja =
        insertXMLbefore(pozycja,
                        '/ksiazka/autorzy/autor[.="Krzysztof J."]',
                        XMLType('<autor>Mariusz M.</autor>'))
```

Funkcja appendChildXML

- Parametry funkcji appendChildXML to:
  - target-data (XMLType) – Dokument XML zawierający dokument, do którego zostaną dodane nowe węzły.
  - parent-xpath (VARCHAR2) – Wyrażenie XPath 1.0, które wskazuje węzły (oprócz atrybutów), wewnątrz których zostaną dodane nowe węzły.
  - child-data (XMLType) – Dane XMLType, które zostaną wstawione jako węzły podrzędne dla węzłów wskazywanych przez parent-xpath
  - namespace (optional, VARCHAR2) – przestrzeń nazw dla parametru parent-xpath.

```
UPDATE ksiazki_or_m_xsd
  SET pozycja =
        appendChildXML(pozycja,
           '/ksiazka/autorzy',
           XMLType('<autor>Witold A.</autor>'));
```

Funkcja deleteXML

- Parametry funkcji deleteXML to:
  - target-data (XMLType) – Dokument zawierający węzły przeznaczone do usunięcia.
  - xpath (VARCHAR2) – Wyrażenie XPath 1.0 które wskazuje węzły przeznaczone do usunięcia. Węzłem usuwanym nie może być węzeł będący korzeniem.
  - namespace (optional, VARCHAR2) – przestrzeń nazw dla parametru xpath.

```
UPDATE ksiazki_or_m_xsd
  SET pozycja =
        deleteXML(pozycja,
                  '/ksiazka/autorzy/autor[position()>3]');
```

>> XQuery <<

- Język zapytań przeznaczony do przetwarzania dokumentów XML – odpowiednich SQL dla RDBMS
- Zapytania XQuery:
  - wykorzystują wyrażenia ścieżkowe zgodnie ze standardem XPath 2.0
  - konstruują dokumenty XML lub ich fragmenty
  - są oparte na wyrażeniach FLWOR
    - FOR
    - LET
    - WHERE
    - ORDER BY
    - RETURN

XQuery "w całości"

```
for $d in doc("depts.xml")//deptno
let $e := doc("emps.xml")//emp[deptno = $d]
where count($e) >= 10
order by avg($e/salary) descending
return
  <big-dept>
    {
    $d,
    <headcount>{count($e)}</headcount>,
    <avgsal>{avg($e/salary)}</avgsal> }
  </big-dept>
```

- Powyższy przykład zawiera wszystkie możliwe klauzule.
  - FOR iteruje po departamentach przypisując zmiennej `$d` kolejne ich numery
  - Dla każdego przypisania zmiennej `$d` klauzula LET przypisuje zmiennej `$e` wszystkich pracowników danego departamentu. Pobiera ich z innego dokumentu.
  - W wyniku działania klauzul FOR i LET otrzymujemy strumień krotek. Każda krotka zawiera parę przywiązanych zmiennych `$d` i `$e`.
  - Klauzula WHERE filtruje ten strumień pozostawiając tylko te krotki, które dotyczą co najmniej 10 pracowników.
  - Klauzula ORDER BY sortuje pozostałe krotki wg średniej płacy w departamencie
  - I wreszcie klauzula RETURN konstruuje nowe elementy `big-dept` indywidualnie dla każdej krotki

Klauzula FOR i LET

- Klauzule FOR i LET wykorzystywane są do tworzenia strumienia krotek. Każda krotka składa się z jednej lub wielu zmiennych.

```
for $s in (<one/>, <two/>, <three/>)
```

```
for $d in document("depts.xml")//deptno
```

Klauzula FOR

- Klauzula FOR może zawierać wiele zmiennych, każdą z odrębnym wyrażeniem.
- W takich przypadkach klauzula FOR iteruje zmiennymi, po każdym elemencie (wyniku) uzyskanym na podstawie wyrażenia.
- W przypadku wykorzystania wielu zmiennych wynikowy strumień wierszy zawiera jeden wiersz, dla każdej kombinacji wyrażeń (iloczyn kartezjański).
- Wynikowa kolejność jest znacząca

```
for $i in (1, 2), $j in (3, 4)
```

```
($i = 1, $j = 3)
($i = 1, $j = 4)
($i = 2, $j = 3)
($i = 2, $j = 4)
```

```
for $i in document("catalog.xml")//item,
    $p in document("parts.xml")//part[partno = $i/partno],
    $s in document("suppliers.xml")//supplier[suppno = $i/suppno]
```

Klauzula FOR i LET (cd)

- Klauzula LET także może zawierać jedną lub wiele zmiennych. Różnica w stosunku do FOR polega na tym, że wiązanie zmiennych odbywa się bez iteracji na podstawie wyrażenia.
- Zmienne z klauzuli LET są dodawane do krotek wygenerowanych przez klauzulę FOR.
- Jeśli klauzuli FOR nie ma, wówczas LET generuje tylko jedną krotkę zawierającą zmienną (zmienne), której zostały przypisane wszystkie elementy spełniające określone wyrażenie.

```
FOR $p IN distinct (document (“bib.xml”)//publisher)
LET $b := document (“bib.xml”)/book[publisher = $p]
```

FOR vs LET

- Mimo iż obie klauzule FOR i LET służą do wiązania zmiennych, sposób w jaki to robią jest zupełnie różny.

```
for $s in (<one/>, <two/>, <three/>)
return <out>{$s}</out>
```

```
<out>
  <one/>
</out>
<out>
  <two/>
</out>
<out>
  <three/>
</out>
```

```
let $s := (<one/>, <two/>, <three/>)
return <out>{$s}</out>
```

```
<out>
  <one/>
  <two/>
  <three/>
</out>
```

Klauzula WHERE

- Opcjonalna klauzula WHERE służy do filtrowania uzyskanych dzięki klauzulom FOR i LET krotek
- Wyrażenie zdefiniowane w klauzuli WHERE jest obliczane dla każdej z uzyskanych krotek.
- Jeśli wyrażenie to jest dla danej krotki prawdziwe, to jest ona poddawana dalszemu przetwarzaniu, np. klauzuli RETURN. W przeciwnym wypadku krotka jest odrzucana.

```
avg(for $x at $i in $inputvalues
    where $i mod 100 = 0
    return $x)
```

```
FOR $p IN distinct (document ("bib.xml")//publisher)
LET $b := document ("bib.xml")/book[publisher = $p]
WHERE count ($b) > 100
RETURN {$p}
```

Klauzula RETURN i ORDER BY

- Klauzula RETURN jest ewaluowana raz dla każdej krotki uzyskanej w wyniku działania wcześniejszych klauzul. Wyniki tej ewaluacji są składają się na rezultat wyrażenia FLOWR.
- Jeśli w zapytaniu nie ma klauzuli ORDER BY wówczas kolejność krotek dostarczanych do klauzuli RETURN jest determinowana przez sposób przetwarzania klauzul FOR i LET.
- Jeśli klauzula ORDER BY istnieje wówczas to ona decyduje o kolejności krotek w strumieniu dostarczanym do RETURN.

Klauzula ORDER BY

- Klauzula ORDER BY zawiera jedno lub wiele wyrażeń.
- Porządek wyznaczany jest poprzez sortowanie wyrażeń występujących kolejno od lewej do prawej.
- Słowa kluczowe występujące w klauzuli ORDER BY:
  - empty least – wartości puste ustawiane są na końcu
  - empty greatest – wartości puste traktowane są jako największe
  - descending – porządek sortowania malejący
  - stable – zachowuje oryginalny porządek krotek w przypadku takiej samej wartości wyrażenia. W przeciwnym wypadku wynika on ze sposobu implementacji

```
for $e in $employees order by $e/salary return $e/name
```

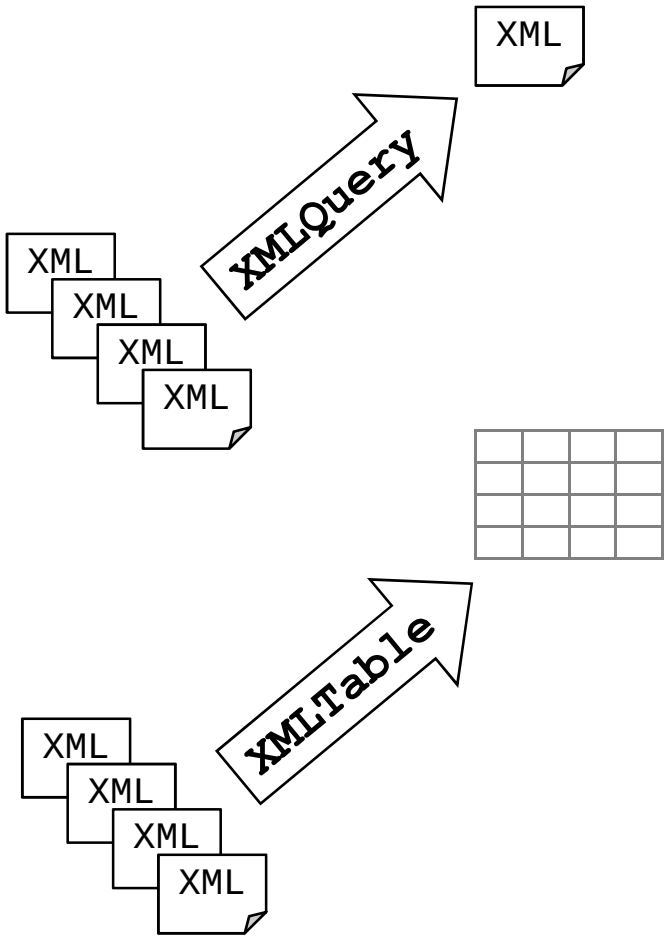
```
for $b in $books//book[price < 100]
order by $b/title
return $b
```

```
for $b in $books//book
stable order by $b/title,
                $b/price descending empty least
return $b
```

Funkcje standardu SQL/XML wykorzystujące XQuery do przetwarzania dokumentów XML

- XMLQuery – funkcja SQL, która konstruuje **obiekt** XMLType, **będący wynikiem zapytania XQuery**
  - Pierwszy parametr to zapytanie XQuery.
  - Wynikiem funkcji może być zarówno pojedynczy element XML lub las elementów.

- XMLTable stanowi pewnego rodzaju rozwinięcie XMLQuery.
  - Zadaniem funkcji XMLTable jest **wyekstrahowanie** z wynikowego dokumentu XML (uzyskanego za pomocą zapytania XQuery) **informacji**, które mogą zostać przedstawione **w postaci relacyjnej**.



Źródła danych dla funkcji wykorzystujących XQuery

- Standardowo źródłem danych dla zapytań XQuery są pojedyncze dokumenty XML lub ich kolekcje, czyli nazwane zbiory dokumentów XML.
- W bazie danych Oracle dokumentem dla zapytań XQuery może być:
  - dokument przechowywany w repozytorium XML
  - dokument znajdujący się w tabeli lub kolumnie typu XMLType
  - dokument zewnętrzny np. dostępny za pomocą protokołu HTTP
- Jako kolekcja dokumentów może natomiast być potraktowany:
  - zbiór dokumentów znajdujący się w określonym katalogu repozytorium XML
  - zbiór wierszy tabeli lub perspektywy, reprezentowanych w postaci obiektów XML o strukturze zbliżonej do mapowania SQL/XML

Przykłady zastosowania funkcji XMLQuery

źródło danych – repozytorium XML

```
SQL> select XMLQuery('for $l in doc("/public/bib.xml")//last
  2                   return $l' RETURNING CONTENT)
  3  from dual;

XMLQUERY('FOR$LINDOC("/PUBLIC/BIB.XML")//LASTRETURN$L'RETURNI
------------------------------------------------------------
<last>Stevens</last>
<last>Stevens</last>
<last>Abiteboul</last>
<last>Buneman</last>
<last>Suciu</last>
<last>Gerbarg</last>
```

```
SQL> select XMLQuery('for $t in collection("/public/shakespeare")/PLAY/TITLE
  2                   return $t' RETURNING CONTENT) wynik
  3  from dual;

wynik
-------------------------------------------------
<TITLE>The Tragedy of Antony and Cleopatra</TITLE>
<TITLE>All&amp;apos;s Well That Ends Well</TITLE>
<TITLE>As You Like It</TITLE>
```

Przykłady zastosowania funkcji XMLQuery

źródło danych – tabele i obiekty XMLType

```
SQL> select XMLQuery('for $n in ora:view("ZESP")/ROW/NAZWA
  2                   return $n' RETURNING CONTENT) wynik
  3  from dual;

WYNIK
--------------------------------------------------------
<NAZWA>ADMINISTRACJA</NAZWA>
<NAZWA>SYSTEMY ROZPROSZONE</NAZWA>
<NAZWA>BADANIA OPERACYJNE</NAZWA>
```

```
SQL> select * from XML_KOL;

XML
--------------------------------
 <ROW>
  <ID_ZESP>10</ID_ZESP>
  <NAZWA>ADMINISTRACJA</NAZWA>
  <ADRES>PIOTROWO 3A</ADRES>
 </ROW>
 <ROW>
  <ID_ZESP>20</ID_ZESP>
  <NAZWA>SYSTEMY ROZPROSZONE</NAZWA>
 </ROW>
. . .

SQL> desc XML_KOL
 Nazwa            NULL? Typ
 ---------------- -------- ------------
 XML                       XMLTYPE
```

```
SQL> select XMLQuery('for $e in $D/ROW/NAZWA
  2                   return $e'
  3  PASSING BY VALUE XML as D
  4  RETURNING CONTENT) wynik
  5  from XML_KOL;

WYNIK
--------------------------------------------
<NAZWA>ADMINISTRACJA</NAZWA>
<NAZWA>SYSTEMY ROZPROSZONE</NAZWA>
<NAZWA>BADANIA OPERACYJNE</NAZWA>
```

Przykłady zastosowania funkcji XMLTable (1/3)


```
SQL> select *
  2  from XMLTable(
  3    'for $i in ora:view("ZESP")/ROW/*
  4     return $i'
  5  COLUMNS B XMLType PATH '.' );

B
----------------------------------------
<ID_ZESP>10</ID_ZESP>
<NAZWA>ADMINISTRACJA</NAZWA>
<ADRES>PIOTROWO 3A</ADRES>
<ID_ZESP>20</ID_ZESP>
<NAZWA>SYSTEMY ROZPROSZONE</NAZWA>
<ID_ZESP>50</ID_ZESP>
<NAZWA>BADANIA OPERACYJNE</NAZWA>
<ADRES>MIELZYNSKIEGO 30</ADRES>
```

```
SQL> select *
  2  from XMLTable('for $i in ora:view("ZESP")/ROW/*
  3                 return $i'
  4          COLUMNS
  5                 LP      FOR ORDINALITY,
  6                 WARTOSC VARCHAR2(20) PATH 'text()'
  7               );

        LP WARTOSC
---------- --------------------
         1 10
         2 ADMINISTRACJA
         3 PIOTROWO 3A
         4 20
         5 SYSTEMY ROZPROSZONE
         6 50
         7 BADANIA OPERACYJNE
         8 MIELZYNSKIEGO 30
```

Przykłady zastosowania funkcji XMLTable (2/3)


```
SQL> select * from
  2   XMLTable('for $e in /ETATY/ETAT
  3             return $e '
  4             passing xmlparse(document(
  5               httpuritype('http://www.../etaty.xml').getclob()))
  6  COLUMNS NAZWA XMLType PATH 'NAZWA',
  7          PLACA_MIN NUMBER(10) PATH 'PLACA_MIN',
  8          PLACA_MAX NUMBER(10) PATH 'PLACA_MAX' )
  9  where PLACA_MIN < PLACA_MAX;

NAZWA                          PLACA_MIN  PLACA_MAX
------------------------------ ---------- ----------
<NAZWA>PROFESOR</NAZWA>               800       1500
<NAZWA>ADIUNKT</NAZWA>                510        750
<NAZWA>ASYSTENT</NAZWA>               300        500
<NAZWA>STAZYSTA</NAZWA>               150        250
<NAZWA>SEKRETARKA</NAZWA>             270        450
<NAZWA>DYREKTOR</NAZWA>              1280       2100

6 wierszy zostało wybranych.
```

www.../etaty.xml:

```
<?xml version="1.0"?>
<ETATY>
  <ETAT>
    <NAZWA>PROFESOR</NAZWA>
    <PLACA_MIN>800 </PLACA_MIN>
    <PLACA_MAX>1500</PLACA_MAX>
  </ETAT>
  <ETAT>
    <NAZWA>ADIUNKT</NAZWA>
    <PLACA_MIN>510</PLACA_MIN>
    <PLACA_MAX>750</PLACA_MAX>
  </ETAT>
  <ETAT>
    <NAZWA>ASYSTENT</NAZWA>
    <PLACA_MIN>300</PLACA_MIN>
    <PLACA_MAX>500</PLACA_MAX>
  </ETAT>
  <ETAT>
    <NAZWA>STAZYSTA</NAZWA>
    <PLACA_MIN>150</PLACA_MIN>
    <PLACA_MAX>250</PLACA_MAX>
  </ETAT>
  <ETAT>
    <NAZWA>SEKRETARKA</NAZWA>
    <PLACA_MIN>270</PLACA_MIN>
    <PLACA_MAX>450</PLACA_MAX>
  </ETAT>
  <ETAT>
    <NAZWA>DYREKTOR</NAZWA>
    <PLACA_MIN>1280</PLACA_MIN>
    <PLACA_MAX>2100</PLACA_MAX>
  </ETAT>
  <ETAT>
    <NAZWA>BEZROBOTNY</NAZWA>
    <PLACA_MIN>0</PLACA_MIN>
    <PLACA_MAX>0</PLACA_MAX>
  </ETAT>
</ETATY>
```

Przykłady zastosowania funkcji XMLTable (3/3)

```
SQL> select NAZWISKO, NAZWA_WALUTY, ROUND(PLACA_POD/KURS,2) PLACA, KOD_WALUTY
  2  from
  3   XMLTable('for $e in /tabela_kursow/pozycja
  4             return $e '
  5             passing xmlparse(document(
  6               httpuritype('http://www.nbp.pl/Kursy/xml/a248z051223.xml').getclob()))
  7             COLUMNS KOD_WALUTY   VARCHAR2(10) PATH 'kod_waluty',
  8                     NAZWA_WALUTY VARCHAR2(30) PATH 'nazwa_waluty',
  9                     KURS         NUMBER(10,2) PATH 'kurs_sredni'),
 10   pracownicy
 11  where NAZWISKO = 'MORZY' ;

NAZWISKO   NAZWA_WALUTY           PLACA KOD_WALUTY
---------- ------------------ -------- ----------
MORZY      dolar amerykański    256,97 USD
MORZY      dolar australijski   351,69 AUD
MORZY      dolar hongkoński    1976,19 HKD
MORZY      dolar kanadyjski     298,56 CAD
MORZY      euro                 216,15 EUR
MORZY      forint węgierski     542,48 HUF
MORZY      frank szwajcarski    336,03 CHF
MORZY      funt cypryjski       123,88 CYP
MORZY      funt szterling       147,69 GBP
MORZY      hrywna ukraińska    1276,92 UAH
MORZY      jen japoński         299,64 JPY
MORZY      korona czeska       6384,62 CZK
MORZY      korona duńska       1627,45 DKK
. . .
```

